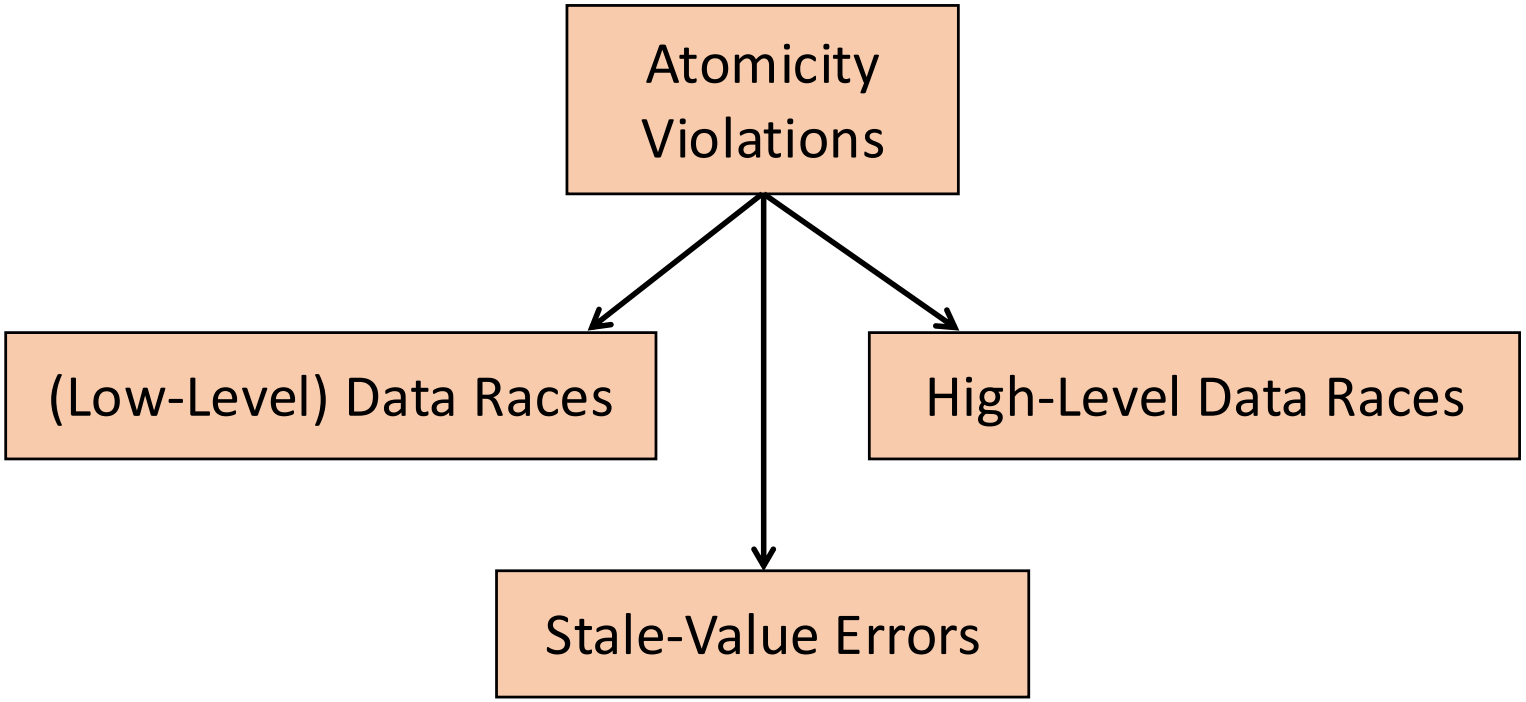
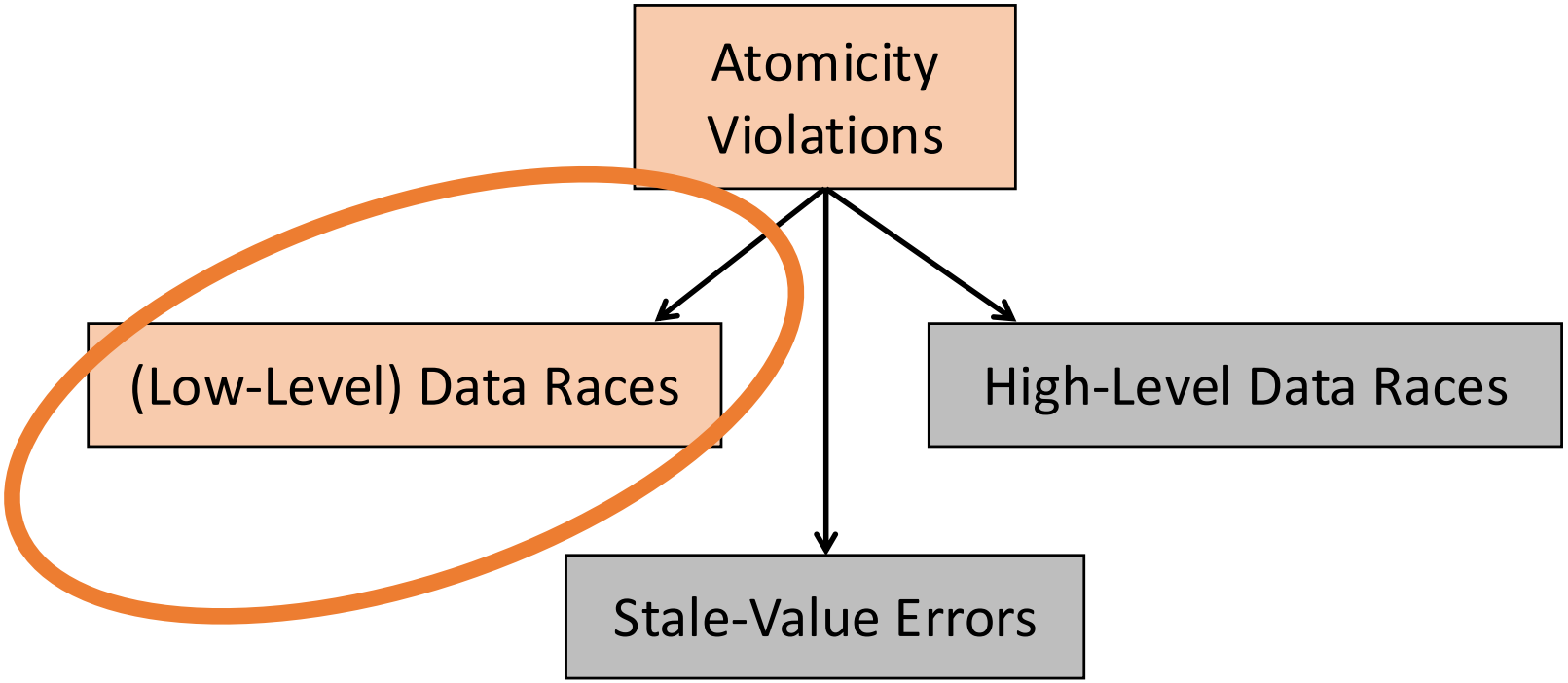
Concurrency Anomalies



Concurrency Anomalies



Lock Definition

- **Lock**: a synchronization object that is either available, or owned (by a thread)
  - Operations: **lock(mu)** and **unlock(mu)**
    - (*We are assuming no explicit initialize operation*)
  - A lock can only be unlocked by its current owner
  - The **lock()** operation is blocking if the lock is owned by another thread

The Happens-before Relation

- *happens-before* (→) defines a partial order for events in a set of concurrent threads
  - In a single thread, *happens-before* reflects the temporal order of event occurrence
  - Between threads, **A** → **B** if **A** is an *unlock* access in one thread, and **B** is a subsequent *lock* access in a **different** thread
    - Assuming the threads obey the sequential specification of the lock , i.e.,

      can't have two successive locks, or two successive unlocks, or a lock in one thread and an unlock in a different thread

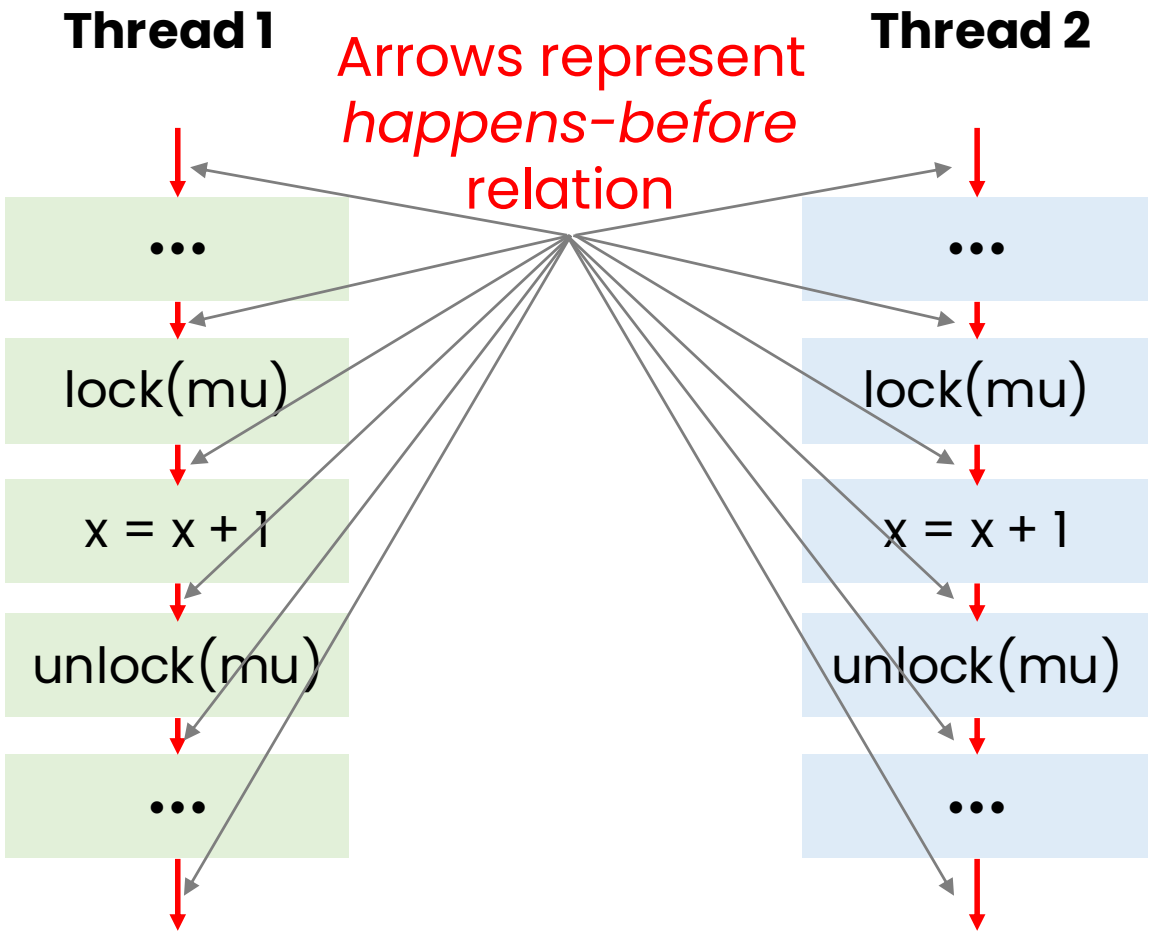
The Happens-before Relation

- Consider **event a** in thread 1 and **event b** in thread 2

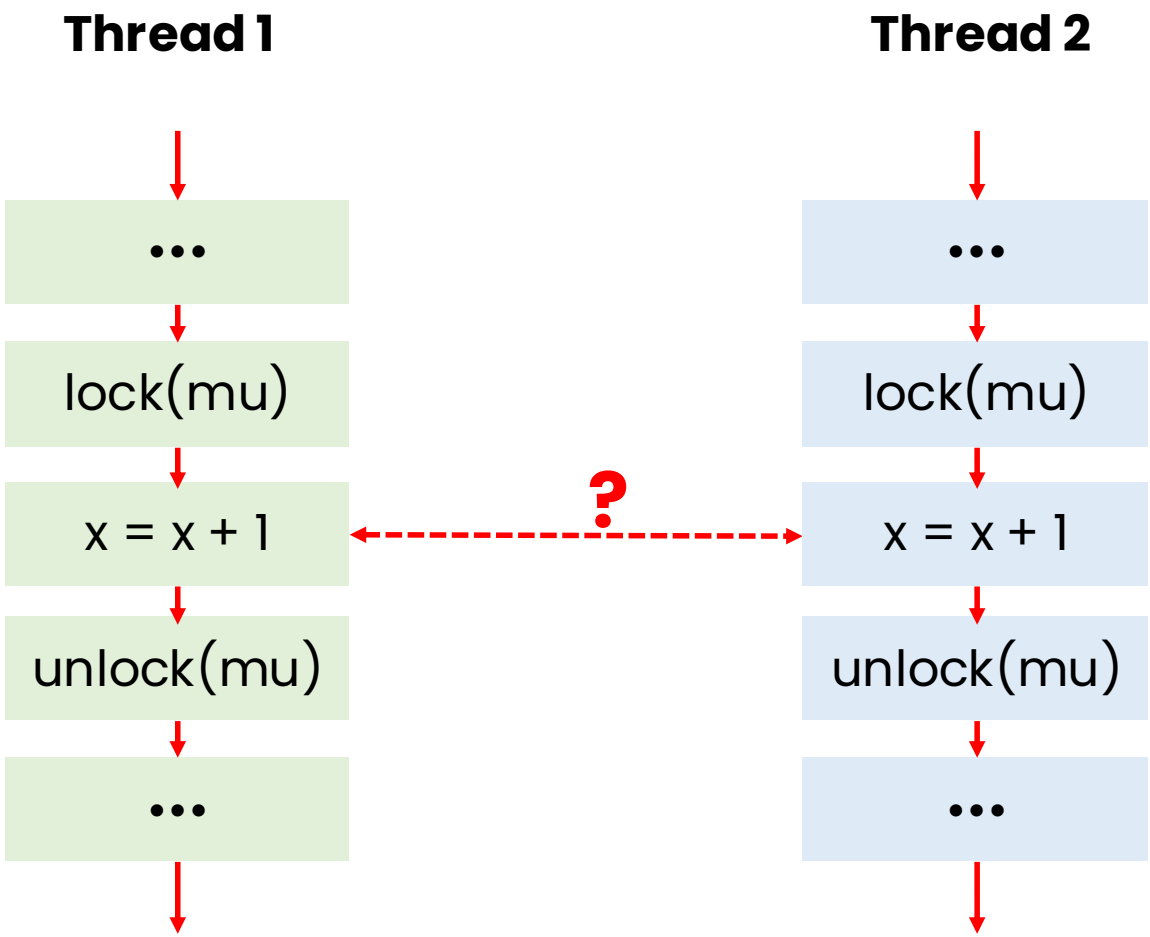
If *a = unlock(mu)* ***and*** *b = lock(mu)*
then $a \rightarrow b$ *(a happens-before b)*

Data races between threads are ***possible*** if accesses to shared variables are not ordered by the *happens-before* relation

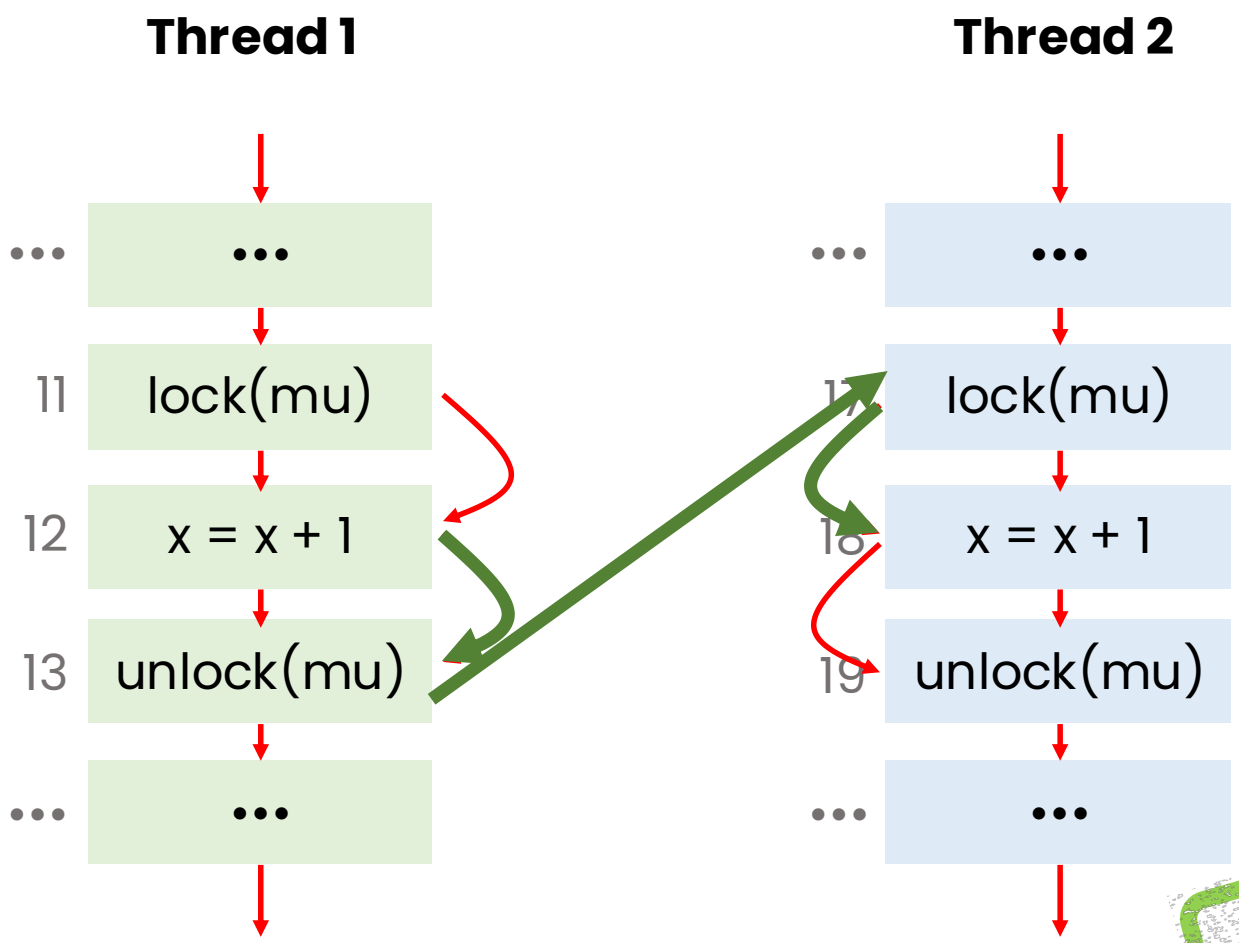
Example 1


Thread 1
Thread 2
Arrows represent *happens-before* relation
...
lock(mu)
x = x + 1
unlock(mu)
...
...
lock(mu)
x = x + 1
unlock(mu)
...

Example 1



Example 1-a



ev 11 → ev 12 → ev 13

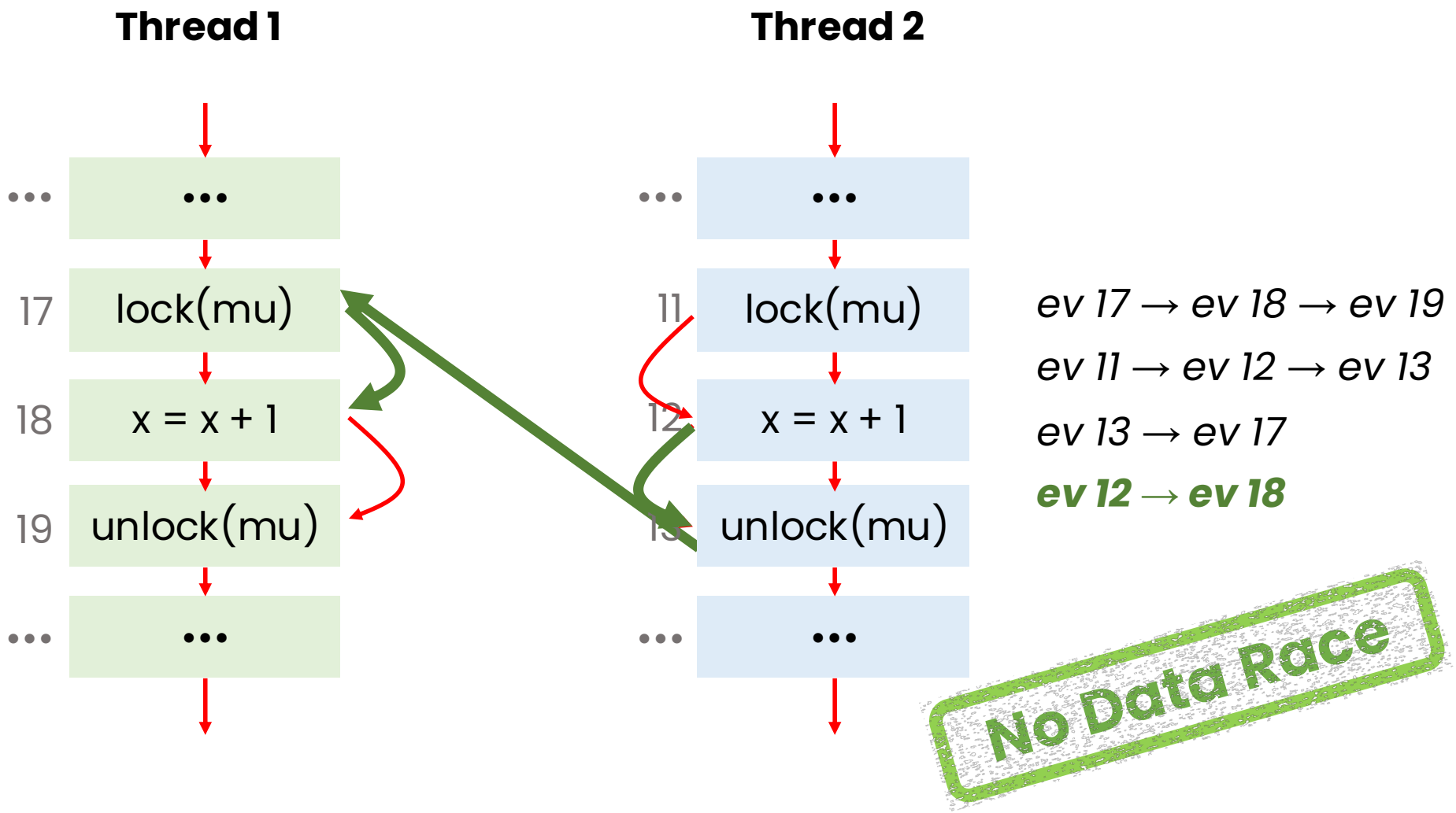
ev 17 → ev 18 → ev 19

ev 13 → ev 17

ev 12 → ev 18

No Data Race

Example 1-b (different ordering)



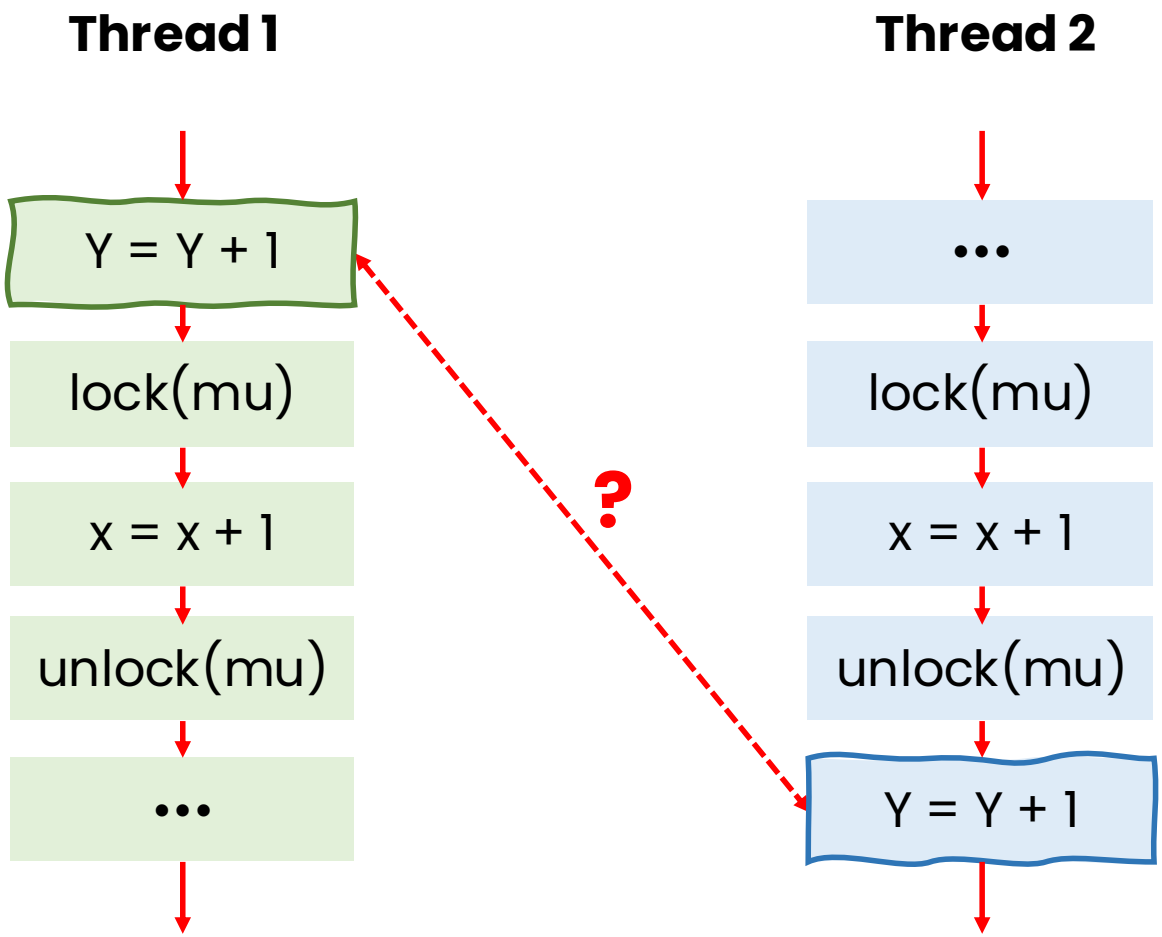
ev 17 → ev 18 → ev 19

ev 11 → ev 12 → ev 13

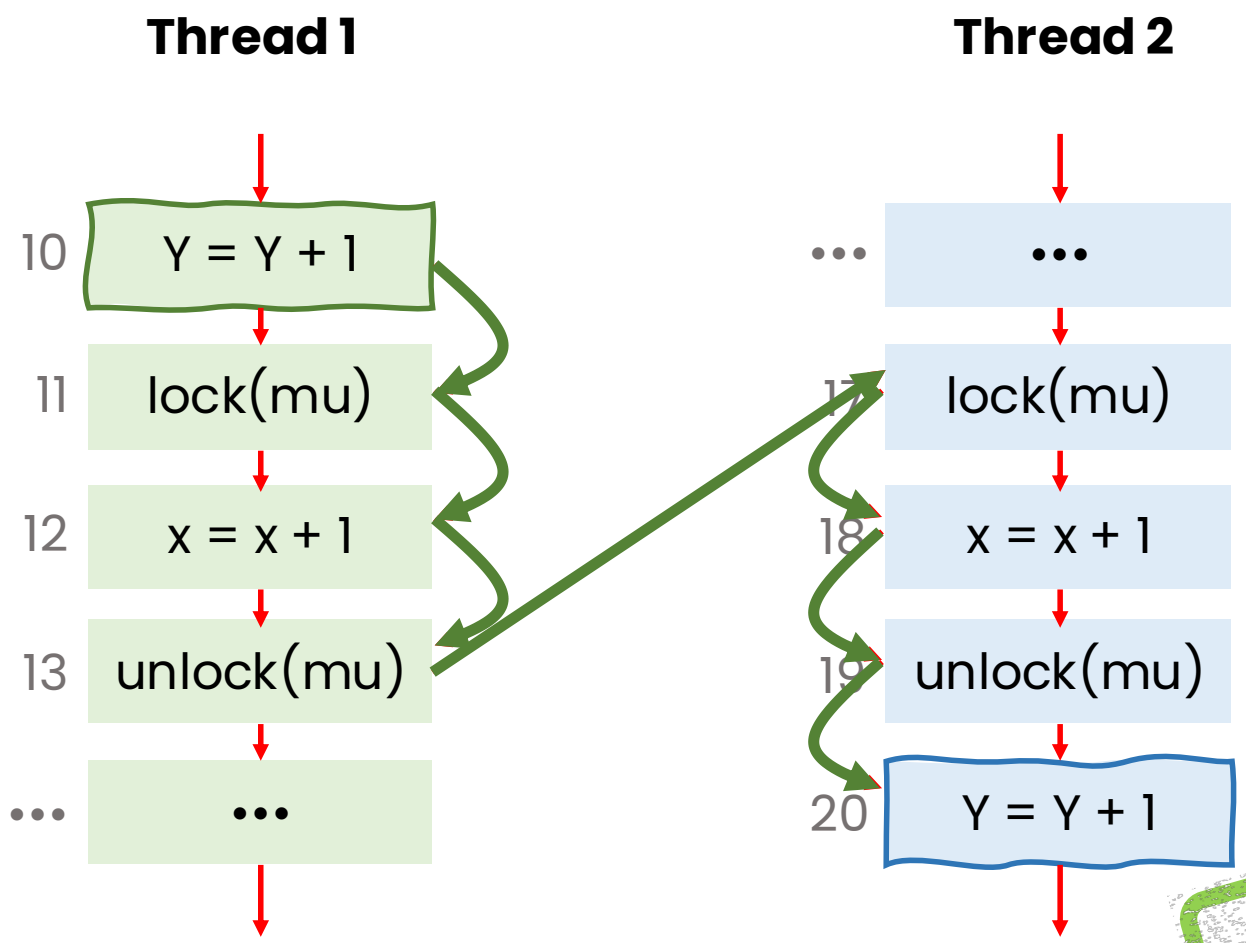
ev 13 → ev 17

ev 12 → ev 18

Example 2


Thread 1
Thread 2
Y = Y + 1
lock(mu)
x = x + 1
unlock(mu)
...
?
...
lock(mu)
x = x + 1
unlock(mu)
Y = Y + 1

Example 2-a



ev 10 → ev 11 → ev 12 → ev 13

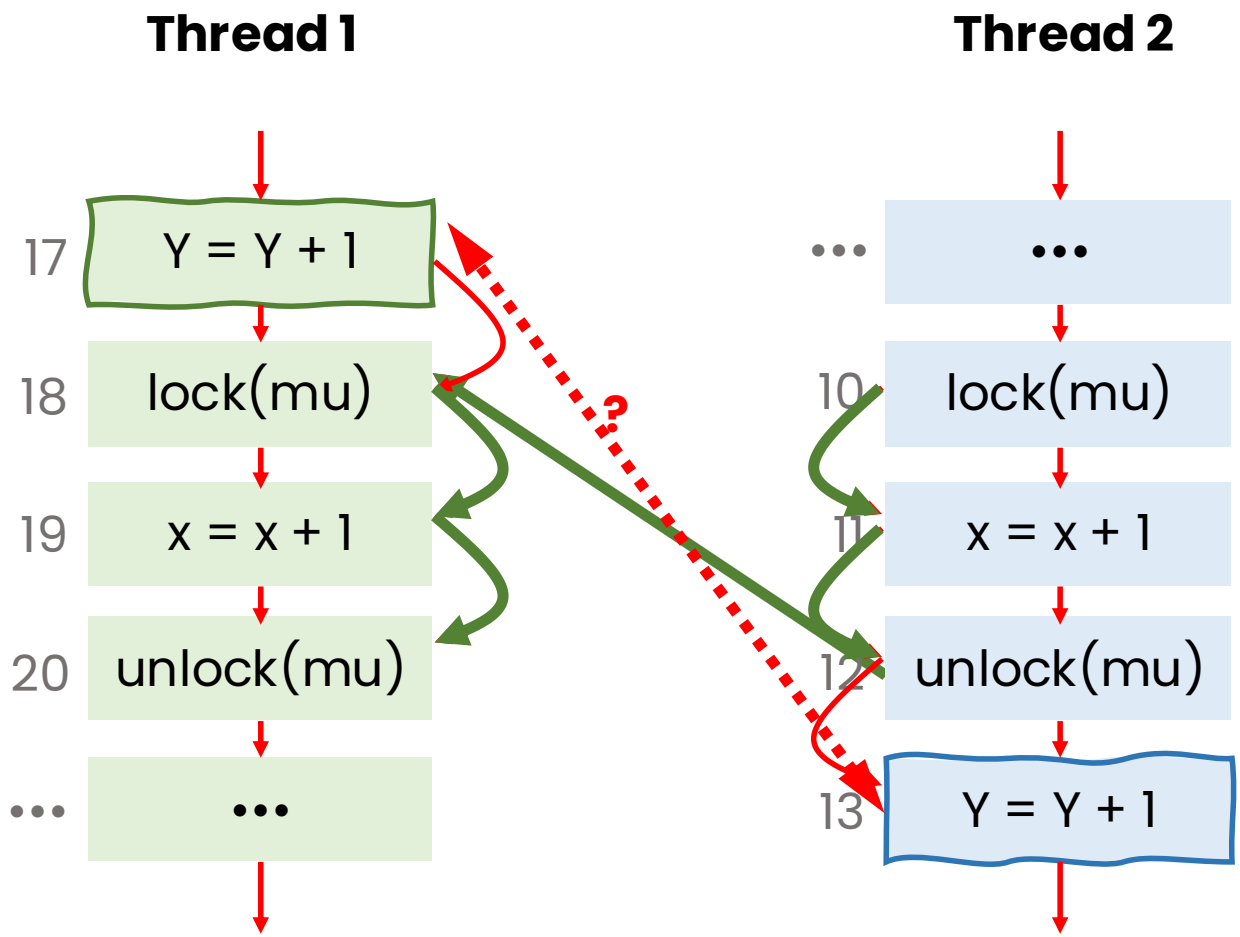
ev 17 → ev 18 → ev 19 → ev 20

ev 13 → ev 17

ev 10 → ev 20

No Data Race

Example 2-b (different ordering)



ev 10 → ev 11 → ev 12 → ev 13

ev 17 → ev 18 → ev 19 → ev 20

ev 12 → ev 18

ev 10 → ev 20

ev 13 ⇆ ev 17